ME5418 — OpenAI Gym Code & Report

Ball on a Two-Axis Tilting Tray

1. Task and MDP Formulation

A solid ball rolls on a square tray (0.3 × 0.3 m) tilting about two axes, mounted on a mobile base that occasionally accelerates. The agent must hold the ball at the tray centre despite that disturbance and an unknown surface friction.

Component	Definition
Action / observation	`Box(-1, 1, (2,))` tilt *increments* (3°/step, tilt clipped to ±15°); `Box(-inf, inf, (18,))` holding the last **3** frames of `[x, y, vx, vy, pitch, roll]`, normalised by tray half-size / 1 m s^{-1} / 15° but deliberately **not clipped**.
Reward / episode end	`r = (1 − d) + 0.5·[dist < 2 cm] − 0.05`$\|a_t - a_{t-1}\|^2$ `− 0.01`$\|a_t\|^2$ with `d = ‖(x,y)‖/d_max`; exactly `−10` on the step the ball leaves the tray, which also terminates the episode. Truncated after 500 steps (10 s at 50 Hz).

Two parameters are **hidden from the observation** (they appear only in `info`), making the task partially observable on two time scales. `c_rr`, the rolling resistance, is constant within an episode (0.005–0.05) and sets a start-up dead zone — a resting ball moves only once `sin(tilt) > c_rr`, i.e. above 0.29°–2.87° — so it is *identifiable* from a few steps of response, which is why three frames are stacked. `a_base`, the base acceleration, is instead 1–3 non-overlapping pulses per episode (0.5–2.0 m s^{-2}, random direction, 0.2–1.0 s); a maximum pulse takes **79 %** of the control authority available at full tilt, and cannot be predicted — only reacted to.

2. Code Explanation

The code is organised so that **one file owns the environment** and everything else only reads it: nothing but `ball_on_tray_gym.py` may change the physical state.

File	Lines	Role
`ball_on_tray_gym.py`	1133	`BallOnTrayEnv(gym.Env)`: dynamics, reward, observation, episode logic, optional realism; `TrayRenderer3D`: **read-only** 3D view used by `render()`.
`controllers.py`	251	PD and LQR baselines (linearised model, Riccati solve), shared by the demo and the evaluation.
`demo.py`	226	The only demo: `--policy random|pd|lqr|both|all`, `--seed`, `--realistic`, `--save-gif`.
`evaluate_baselines.py`	642	Batch evaluation, no rendering: condition grid → CSV + survival heat maps in `results/`; `--realism-sweep`.
`tests/test_{env, renderer_3d, realism}.py`	303 / 207 / 327	14 + 7 + 8 tests; all 29 pass. `test_realism.py` replays `golden_v1.npz`, recorded before the realism parameters existed.

Dynamics. Per axis, for a solid sphere rolling without slipping, `a_drive = (5/7)·(g·sin(tilt) − a_base)` and `a_rr = (5/7)·c_rr·g`, where 5/7 is `1/(1 + I/mr²)`. Each 20 ms control step is integrated as **4 semi-implicit Euler sub-steps** with tilt (by default) and `a_base` held constant, aborting the moment the ball crosses the edge so termination cannot be missed inside a step. Rolling resistance is a speed decrement that can never reverse the velocity, plus a dead zone for a resting ball; without those guards a constant friction deceleration makes a nearly stationary ball oscillate.

Optional realism (off by default) relaxes the proposal's ideal sensor and actuator. `pos_noise_std` / `vel_noise_std` add Gaussian noise to the *observation only*, from its own generator seeded by the reset seed, so a seed keeps its episode; `actuator_tau` lets the actual tilt follow the command through a first-order lag, `tilt += (1 − e^(−h/τ))·(tilt_cmd − tilt)` per sub-step; `action_delay_steps` applies each action *k* steps late (the reward still charges the action given). With the defaults the environment matches the pre-realism snapshot to 10^{-9}.

Visualisation. The renderer creates every matplotlib artist once and thereafter only updates artist data. `"human"` mode drops frames when the machine cannot sustain 50 fps, so playback stays real-time; `"rgb_array"` draws off-screen with no GUI backend and drives the GIF recording. The 3D tray rotation is *derived* from the environment's sign convention rather than guessed: requiring $a_x \propto g\cdot\sin(\text{pitch})$ and $a_y \propto g\cdot\sin(\text{roll})$ fixes the third row of `R`, which with no yaw determines `R` uniquely. Angles are drawn to scale.

Demos and baselines. `demo.py` plays every selected policy **under the same seed**, so they face the same start position, `c_rr` and disturbance schedule — a controlled comparison, not unrelated clips. Both *deterministic* baselines are deliberately given **no privileged information**: each reads only the newest observation frame, so neither sees `c_rr` or `a_base` nor has any feed-forward term for the disturbance, which keeps them comparable with a future RL agent.

PD (`controllers.py`) de-normalises the newest frame and runs

```
tilt_target = clip(−Kp·pos − Kd·vel, ±15°)      action = clip((tilt_target − tilt) / 3°, −1, 1)
```

with `Kp = 6` rad m^{-1}, `Kd = 1.5` rad s m^{-1} — an outer PD sets a target attitude and an inner proportional loop chases it, because the action is a tilt *increment* rather than a tilt.

LQR (`controllers.py`) makes that structure explicit instead of hand-tuning it. The axes are exactly decoupled (pitch drives x, roll drives y), so one single-axis design serves both, and since the action is a tilt increment the tilt is itself a state: `z = [x, v, θ]`^{T}. Deriving `A` and `B` required reading our own integrator carefully — semi-implicit sub-stepping puts the position coefficient at `(N+1)/(2N) = 0.625`, not the `0.5` of an exact double integrator — after which the model reproduces one environment step to ~10^{-7} m near level. Iterating the discrete Riccati equation to convergence (128 steps) gives `K = [95.7, 17.6, 11.2]` and closed-loop poles at |z| = 0.877, 0.877, 0.537, which the demo checks for stability itself.

These rollouts also **validate the reward function** — the most error-prone part of an RL environment, and the hardest to debug through an agent. Over the same 100 seeds at the full randomisation ranges (`evaluate_baselines.py`):

Policy	Return	Episodes completed	Reward / step
Random	7.6 ± 12.3	0 / 100 (fell after 39.7 ± 19.2 steps)	0.191
PD	712.6 ± 11.7	**100 / 100**	1.425
LQR	**724.4 ± 8.3**	**100 / 100**	**1.449** (96.6 % of the 1.5 ceiling)

Policies that visibly do the right thing score two orders of magnitude above one that visibly does not, and no reward term swamps the others. LQR beats PD modestly on the mean but clearly on the spread (29 % smaller) and the steady-state error (2.7 vs 4.5 mm); after the first 1.5 s, the worst pulse-driven excursion is 3.2 cm (PD) / 2.3 cm (LQR). A fixed grid (`c_rr` $\in$ {0.005, 0.0275, 0.05} × pulse $\in$ {0, 0.5, 1, 2, 3} m s^{-2}) finds the edge: in-range cells all keep ≥ 98 % survival (the 7 failures in 2400 episodes all during 2 m s^{-2} pulses), but 3 m s^{-2}, needing 17.8° > 15° of tilt, cuts it to 63 % / 60 % on the least-damped tray.

Realism sweep (`--realism-sweep`, full range, same gains, seeds 0–99): survival % / return % of max (bold: higher return).

	ideal	noise 2 mm, 0.03 m/s	lag 0.05 s	lag 0.10 s	delay 1 step	delay 2 steps	noise + lag 0.05 + delay 1
PD	100 / 95.0	100 / 85.0	**100 / 95.0**	**97 / 89.4**	100 / 91.2	**100 / 91.6**	**99 / 87.8**
LQR	**100 / 96.6**	**100 / 90.6**	98 / 93.5	84 / 71.4	**99 / 95.3**	98 / 90.7	97 / 83.7

Noise costs return, not survival: the velocity terms turn it into ~30 action sign changes per second, charged by the smoothness penalty. **Lag defeats the fixed gain**, the optimal one most: LQR loses 16 % of episodes at τ = 0.10 s (PD 3 %) and falls below PD with noise + lag + delay.

3. Existing Codes / Libraries

What we took. A deliberately thin slice of `gym==0.26.2`: the `gym.Env` base class, `spaces.Box`, `super().reset(seed=...)` and the resulting `self.np_random` generator for all episode sampling, the `metadata` dict, the 0.26 five-tuple `step` convention, and `gym.utils.env_checker.check_env` in the tests. `numpy` does the arithmetic, `matplotlib` supplies the drawing primitives (`mplot3d` for the 3D view, pyplot for heat maps), and `PIL`, a matplotlib dependency, writes the GIFs.

What we wrote ourselves. No robotics simulator is involved — no PyBullet, MuJoCo, Gazebo or Isaac — and no control library either. The dynamics, the integrator, the rolling-resistance model, the disturbance sampler, the noise / lag / delay models, the reward, the 3D visualisation, the evaluation and the Riccati solver are written from scratch. We preferred this to a physics engine because the system is low-dimensional enough to integrate exactly as specified, which keeps the dynamics auditable: our tests assert the measured acceleration against the closed-form `(5/7)·g·sin(tilt)` to a relative tolerance of 10^{-9}, which a black-box engine would not permit. `scipy.linalg.solve_discrete_are` would have returned the gain in one call, but a dependency for a single 3×1 solve seemed a poor trade.

Setup. Developed on Windows 11, conda 26.7.1, Python 3.7.16; `README.md` has the setup and every option.

4. Reflections / Lessons Learned

Simplifications. The manipulator is not modelled: by default the tray reaches the commanded tilt instantly, so the agent commands tilt, not joint torques (the optional lag and delay mimic the arm's response, not its kinematics or torque limits). The ball's radius is ignored geometrically (it falls when its centre crosses the edge), and we assume rolling without slipping throughout — defensible at 15° maximum tilt, but it would break on a steeper or more slippery tray.

Changes from the proposal. The proposal's friction coefficient became a rolling-resistance coefficient `c_rr`, since the ball rolls without slipping; the distance reward $-\|(x, y)\|$ became the normalised $1 - d$, which also acts as a per-step survival term; and disturbance pulses are kept non-overlapping. The second baseline is a hand-tuned PD controller rather than the proposal's grid-searched fixed gain, which is deferred to the final comparison against the RL agent.

What coding changed about the design. Five decisions were made at the keyboard, not in the proposal. (i) *Frame stacking*: with a single frame, `c_rr` and an active disturbance are unobservable, so a Markov policy could not compensate for them. The *controller* state `[x, v, θ]` is already complete in one frame — the stacking exists purely for the hidden parameters, which is why neither baseline needs more than the newest frame. (ii) *Not clipping the observation*: we first clipped to [−1, 1] and realised this destroys the information in the very step where the ball leaves the tray. (iii) The two rolling-resistance guards in §2, found only by watching a stationary ball jitter in the (since retired) 2D view — an argument for building the visualisation early rather than last. (iv) Writing the LQR model forced us to re-read our own integrator and find the `0.625` coefficient: modelling your own simulator turns out to be an effective way to audit it. (v) *Realism without moving the baseline*: a snapshot recorded first proves the defaults unchanged, and a separate noise generator keeps each seed's episode, so paired comparisons stay valid.

What we would change for the final version. The full-range table says the ideal environment is **too easy to be a convincing test bed for RL**: two fixed-gain controllers survive 100/100 and PD trails optimal LQR by only 1.7 % of return. The grid and the sweep show where it stops being easy: pulses beyond the tilt authority, and actuator lag. Noise, lag and delay are now in place (off by default, so the ideal task stays as a first curriculum stage); still to do: a wider friction range, a continuous rather than pulsed disturbance, a smaller goal radius, a longer episode so long-horizon credit assignment matters, and a moving goal, turning regulation into tracking. The case for RL here is not raw performance but *adaptation*: LQR is optimal for the model it was given, which contains neither `c_rr`, `a_base` nor the lag, and the sweep already shows it degrading faster than PD once the lag is unmodelled; a policy that infers them from the frame stack should eventually win.